SMART PARKING SYSTEM

For Smart Cities

Project Documentation

Built with Python Flask + IoT Simulation

Submitted By
Samarth More

Project Name	Smart Parking System
Technology	Python Flask, HTML, CSS, JavaScript
Architecture	IoT + REST API + Web Dashboard
Total Slots	40 Parking Slots
Version	1.0
Year	2024

1. Introduction

A Smart Parking System is an IoT-powered solution designed to optimize parking space utilization in urban environments. By embedding sensors into parking slots and connecting them to a centralized management system, drivers can locate available spaces in real time — reducing traffic congestion and improving the overall city mobility experience.

This project implements a fully functional Smart Parking System using Python Flask as the backend framework, with a responsive web-based dashboard that simulates real-world IoT sensor behavior. The system manages 40 parking slots, two automated gates, a live entry display board, an activity log, and vehicle counters.

1.1 Problem Statement

Urban parking is one of the biggest challenges in modern smart cities. Drivers waste significant time searching for available parking, contributing to:

- Increased traffic congestion in city centers
- Higher fuel consumption and carbon emissions
- Driver frustration and reduced productivity
- Unmonitored and mismanaged parking facilities

1.2 Proposed Solution

The Smart Parking System addresses these challenges by providing:

- Real-time monitoring of all 40 parking slots
- Automated gate control on vehicle entry and exit
- A live entry display board showing available and occupied slots
- IoT sensor simulation for realistic data flow
- A complete REST API backend for integration with external systems

2. Objectives

The primary objectives of the Smart Parking System project are:

1. Design and develop a real-time IoT-based parking management web application.
2. Implement automated gate control for vehicle entry and exit detection.
3. Provide a live dashboard showing slot occupancy, vehicle counts, and activity logs.
4. Simulate IoT sensor data to mimic real parking sensor behavior.
5. Build a RESTful API backend capable of integration with real hardware sensors.
6. Display a digital entry board showing available and occupied slot counts.

3. System Architecture

The Smart Parking System follows a three-tier architecture that mirrors a real-world IoT deployment as described in the IBM Watson IoT + Node-RED architecture diagram.

3.1 Architecture Layers

Layer	Component	Technology
IoT Layer	Parking Slot Sensors / Gate Sensors	Python Simulation
Backend Layer	REST API + Business Logic	Python Flask
Frontend Layer	Web Dashboard + Display Board	HTML, CSS, JavaScript

3.2 Data Flow

The data flow through the system follows this sequence:

7. IoT sensors (simulated via Python) detect vehicle presence in parking slots.
8. Sensor data is sent to the Flask REST API via HTTP POST requests.
9. Flask backend updates the parking state and returns updated statistics.
10. The JavaScript frontend polls the API every 5 seconds and re-renders the dashboard.
11. The entry display board updates in real time showing FREE and TAKEN counts.
12. Gates open and close automatically based on vehicle entry and exit events.

3.3 Project File Structure

```
smart_parking/
├── app.py                  ← Flask backend + REST API
├── requirements.txt        ← Python dependencies
├── README.md
```

```
├── templates/
│   └── index.html          ← Main dashboard UI
└── static/
    ├── css/
    │   └── style.css       ← Dark theme stylesheet
    └── js/
        └── app.js          ← Live updates, animations, pie chart
```

4. Features

4.1 Real-Time Slot Monitoring

The dashboard displays all 40 parking slots in a 10x4 grid layout. Each slot is color-coded:

- Green slot — Available (empty)
- Red slot — Occupied (filled)

Slots can be toggled manually by clicking on them, simulating a direct sensor override. This is useful for testing and manual management scenarios.

4.2 Automated Gate Control

The system features two animated gates — Entry Gate and Exit Gate. Each gate has a barrier arm that animates open and closed when triggered. Gate behavior:

- Entry Gate opens when a vehicle enters — allocates the next available slot
- Exit Gate opens when a vehicle exits — frees the slot and updates counters
- Gate status indicators show Open (green dot) or Closed (red dot) in real time
- Gates auto-close after 2.4 seconds, mimicking real barrier timing

4.3 Entry Display Board

A digital display board styled in green-on-black monochrome (mimicking a real LED parking sign) shows:

- Total capacity (40 slots)
- Number of FREE slots
- Number of TAKEN slots
- System status: OPEN or FULL — NO ENTRY

4.4 Vehicle Counter

The system maintains a running count of all vehicle movements during the session:

Counter	Description	Reset On
Entered Today	Total vehicles that entered the parking area	System Reset
Exited Today	Total vehicles that left the parking area	System Reset
Available	Current free slots (live count)	Auto-updates
Occupied	Current filled slots (live count)	Auto-updates

4.5 IoT Sensor Simulation

The IoT Simulate button mimics the behavior of real parking sensors. Each click fills the next available empty slot in sequential order (Slot 4, 5, 6...), adding an activity log entry tagged as [IoT Sensor]. This allows demonstration of real-world sensor-driven occupancy updates without physical hardware.

4.6 Activity Log

A timestamped activity log records all parking events in real time:

- ENTRY — Vehicle entered (manual or IoT sensor)
- EXIT — Vehicle exited
- SYS — System events (reset, startup, manual toggles)

4.7 Live Pie Chart

A canvas-based donut pie chart updates in real time showing the visual ratio of Available vs Occupied slots, with the occupancy percentage displayed in the center.

5. REST API Reference

The Flask backend exposes a complete REST API. All endpoints return JSON responses and can be integrated with real IoT hardware, mobile apps, or third-party systems such as IBM Watson IoT Platform or Node-RED.

Method	Endpoint	Description
GET	/api/status	Returns full system state: stats, all slots, last 20 log entries
POST	/api/entry	Triggers vehicle entry — allocates next free slot, opens entry gate
POST	/api/exit	Triggers vehicle exit — frees first occupied slot, opens exit gate
POST	/api/toggle_slot/<id>	Toggles a specific slot between empty and filled
POST	/api/simulate_random	IoT simulation — fills the next empty slot in order
POST	/api/gate_close	Closes both gates (called automatically after 2.4s)
POST	/api/reset	Resets entire system to initial state (3 occupied, 37 free)

5.1 Sample API Response — /api/status

```
{
  "stats": {
    "total": 40,
    "available": 37,
    "occupied": 3,
    "occupancy_pct": 7.5,
    "entered_today": 1,
    "exited_today": 0,
    "status": "OPEN"
  },
  "slots": [
    {"id": 1, "status": "filled"},
    {"id": 2, "status": "filled"},
    ...
  ],
  "log": [
    {"time": "11:30:04", "type": "enter", "message": "Vehicle entered — Slot 4
allocated"}
  ]
}
```

5.2 Running the Application

```
# Step 1 — Install Python (3.8 or higher)
python --version

# Step 2 — Navigate to project folder
cd smart_parking

# Step 3 — Install Flask
pip install -r requirements.txt

# Step 4 — Start the server
python app.py

# Step 5 — Open browser
http://127.0.0.1:5000
```

6. Real-World Scenarios

Scenario 1 — Rush Hour Navigation

A driver enters the city centre during rush hour. Using the smart parking app connected to this system, they input their destination. The system checks the /api/status endpoint and navigates the driver to the nearest available parking slot, minimizing search time and reducing traffic congestion.

Scenario 2 — Commuter Route Planning

A commuter leaving work checks the smart parking dashboard. They receive real-time updates on parking availability near their home, enabling them to plan their route and avoid congested areas. The live slot grid shows exactly which slots are free.

Scenario 3 — Automated Payment & Exit

After parking in the smart parking facility, the driver's session is tracked from the moment the entry gate opens. Upon exit, the barrier lifts automatically via the /api/exit endpoint, eliminating the need for manual ticket validation or cash transactions.

7. Technologies Used

Technology	Purpose	Version
Python	Core programming language	3.8+
Flask	Web framework and REST API server	3.0+
HTML5	Dashboard structure and layout	5
CSS3	Dark theme styling and animations	3
JavaScript	Live updates, gate animation, charts	ES6+
Canvas API	Real-time donut pie chart	HTML5
Font Awesome	Icons for UI elements	6.5

8. Conclusion

The Smart Parking System successfully demonstrates how IoT technology can be leveraged to solve real urban mobility challenges. By combining a Python Flask REST API with a responsive real-time web dashboard, the system provides:

- Accurate real-time slot occupancy monitoring across 40 parking bays
- Automated gate management with smooth barrier animations
- A live entry display board for driver guidance at the facility entrance
- Complete vehicle entry and exit tracking with session counters
- IoT sensor simulation enabling realistic demonstrations without hardware
- A fully documented REST API ready for integration with IBM Watson IoT, Node-RED, or any mobile application

This project serves as a strong foundation for a production-ready smart city parking solution. Future enhancements could include license plate recognition, mobile app integration, payment processing, and real hardware sensor connectivity via MQTT protocol.